Homework - 9

①

```
void insertion_sort (int n, keytype S[J)
{
        index i,j;
        keytype x;
        for (i=2; i<=n, i++){
            x = S[i];
            j = i-1;
            while ( S[j] >x) { // j>0 is removed since S[0] = -inf.
                S[j+1] = S[j];
                j--;
            }
            S[j+1] = x;
        }
}
```

→ Exact time complexity :

i goes from $2 \rightarrow n \quad (n-1)$

j goes from $i-1 \rightarrow 0 \quad (i-1+1 = i \text{ times})$

In the worst case scenario when the array is reversed, we need to traverse from $i = 2$ to n.

$$\text{Total steps} = W(n) = \sum_{i=2}^{n} i = 2 + 3 + \dots n = \frac{n(n+1)}{2} - 1$$

$$W(n) = \frac{n^2+n-2}{2}$$

$$O\left(\frac{n^2+n-2}{2}\right) = O(n^2)$$

→ Time complexity is exacty same as the original one. Both algorithms are $O(n^2)$.

→ The new version is better than the original version because the original version evaluates a compound condition $j>0$ && $S[j]>x$ which is two steps but the newer version only evaluates $S[j]>x$ which is one step.

(2) Let permutation $P = [P_1, P_2, \ldots, P_n]$ and it transpose or reverse be $P' = [P_n, P_{n-1}, P_{n-2}, \ldots P_1]$.

- P_i, P_j be a pair of distinct elements in the permutation ($P_i < P_j$)
- In the original permutation, these two should appear in order. There are 2 possibilities:

(i) P_i comes befor P_j : this is their natural order, so not an inversion in P. In P', P_j comes before P_i because it is exact reverse of P and this is an inversion in P'.

(ii) P_j comes before P_i : this is an inversion in P. In P', P_i comes befor P_j and P' is not an inversion.

- We have one inversion for every possible pair of distinct elements from P and P'.
- Number of inversions is equal to number of ways you pick 2 elements from n elements which is

$$^nC_2 = \binom{n}{2} = \frac{n(n-1)}{2}$$

∴ Total sum of inversions in P and P' $= \frac{n(n-1)}{2}$

→ $P = [2, 5, 1, 6, 3, 4]$, $n = 6$

$P' = [4, 3, 6, 1, 5, 2]$

Inversions in P

2, 1 → 1 inversions

5, {1, 3, 4} → 3 inversions

6, {3, 4} → 2 inversions

6 inversions

Inversion in P'

4, {1, 2, 3} → 3

3, {1, 2} → 2

6, {1, 5, 2} → 3

5, 2 → 1

9 inversions

Total sum in P & P' = 6 + 9 = 15

n = 6 $\quad \frac{6(6-1)}{2} = 15$ → Both match too

(3)

a) We can use selection sort because it performs only O(n) swaps. Since records are long that is expensive to move but keys are short that is cheap to compare, the O(n) swaps in selection sort minimizes expensive record movements.

b) We can use heapsort because it has O(nlogn) in all cases and uses O(1) extra space. Quicksort's worst case is $O(n^2)$ and mergesort's extra space is O(n) which is expensive for 45,000 records.

c) We can use insertion sort as it runs in O(n) time on nearly sorted data because it only needs to shift few elements.

d) We can use quicksort because it has an average time complexity of O(nlogn). Since, worst-case performance is not critical, $O(n^2)$ is an acceptable time complexity in few cases.

(4) Given a permutation of numbers 1 to n 'arr'.

```
def linear_sort(arr) :
    n = len(arr)
    aux_arr = [0] * n
    for i in range(n) :
        aux_arr[arr[i] - 1] = arr[i]
    for i in range(n) :
        arr[i] = aux_arr[i]
    return arr
```

aux_arr : auxiliary array used for sorting.

Each for loop runs n times $O(n+n) = O(2n) = \underline{\underline{O(n)}}$.